Practice Exam (Unofficial)

April 6, 2025

Note: all of these were questions made by me (and I haven't seen your exam), so they may not be indicative of the real exam. I believe your exam has 10 questions in part 1, but I included 20 here to cover a variety of topics. Good luck!

Part 1: Conceptual Questions

1. What is the basic Linux command to list the files and directories in the current working directory, including hidden files?

2. You have modified a file named 'feature.c'. What are the three basic Git commands needed to stage this change, commit it with the message "Implement new feature", and push it to the remote repository (origin/main)? (Assume you are already on the "main" branch and it already points to origin.

3. Consider the following C code:

```c
int main() {
    float result = 5 / 2;
    printf("Result: %f\n", result);
    return 0;
}
```

   What will be printed and why? Write any modifications necessary to produce the desired result.

4. What is the data type of the variable 'ptr' in the following declaration?

```c
char message[] = "Hello";
char *ptr = message;
```

5. Examine the C code below. Is the function "modifyValue" demonstrating pass-by-value or pass-by-reference? Explain briefly. What is the printed output?

```c
#include <stdio.h>

void modifyValue(int *x) {
```

```
    *x = *x + 10;
}

int main() {
    int num = 5;
    modifyValue(&num);
    printf("Num: %d\n", num); // output?
    return 0;
}
```

6. What character signifies the end of a string in memory? Does the value returned by `strlen(s)` include this character?

7. Consider this hash function: `hash(key) = key % 11`. If you are inserting keys 15, 25, and 35 into a hash table of size 11 using this function, what index will each key map to?

8. What is the potential problem in the following C code? Suggest a safer alternative from the string library (`string.h`).

```
char dest[10];
char src[] = "This is a long string !!!!!";
strcpy(dest, src);
```

9. Explain the difference between a global variable and a local variable in C regarding their scope and lifetime.

10. What does the 'static' keyword do when used with a local variable inside a function in C? What happens to 'count' between calls to counter()?

```
void counter() {
    static int count = 0;
    count++;
    printf("Count: %d\n", count);
}
```

11. You have three files: 'main.c', 'utils.c', and 'utils.h'. 'main.c' includes 'utils.h' and calls functions defined in 'utils.c'. What is the command you would use in the terminal to compile these files into an executable named 'myprogram'? Assume the compilation is a success and you're in the same directory as the program. What would you type in the terminal to execute the program?

12. Identify the error in the following code. How would you fix it?

```
typedef struct {
    int id;
    char name[50];
} Record;
```

```c
int main() {
    Record *recPtr = (Record *) malloc(sizeof(Record));
    if (recPtr == NULL) return 1; // error allocating

    recPtr.id = 101;
    recPtr.name = "record1";

    free(recPtr);
    return 0;
}
```

13. What is the primary difference between ‘malloc‘ and ‘calloc‘ regarding dynamic memory allocation and initialization?

14. Briefly describe the difference between a singly linked list and a doubly linked list.

15. What is the basic GDB command to set a breakpoint at the beginning of a function named ‘calculate_sum‘?

16. Consider the following macro definition:

```c
#define SCALE(x, factor) x * factor
```

What is the value of ‘result‘ after the following code? If there are any issues, explain how to fix them.

```c
int result = SCALE(5 + 2, 10);
```

17. In the following C code, describe what `arr` actually is. Assume that `sizeof(double *)` returns 8 bytes. Then, what would sizeof(arr) return?

```c
double *arr[20];
```

18. How many lines of "hi" are printed when executing the following:

```c
#include <stdio.h>
int main() {
    for (int i = 1; i <= 2; i++) {
        printf("hi\n");
        for (int j = 0; j < i; j++) {
            printf("hi\n");
        }
    }
    return 0;
}
```

19. Suppose you have a pointer ‘data_buffer‘ that points to a block of memory that was previously allocated with ‘malloc‘ to hold ‘current_size‘ characters. You now need to resize this block to hold ‘new_size‘ characters. Write a single C statement to do this efficiently. Let the memory address be assigned back to ‘data_buffer‘. (You don’t need to handle error checking).

```c
// Assume: size_t current_size; size_t new_size;
char *data_buffer = (char *) malloc(current_size);
// (your line here to resize)
```

20. In the context of calculations involving floating-point numbers, what is meant by the term "catastrophic cancellation"? Briefly explain the situation where it is most likely to occur.

Part 2: Data Structure Implementation

In this part, you will implement a basic stack data structure using a singly linked list in C. The stack should store integer values.

a)

Briefly discuss one main advantage and one main disadvantage of implementing a stack using a linked list compared to using a fixed-size array.

b)

Define the necessary C structs using ‘typedef‘. You will need:

- A structure ‘StackNode‘ to represent a node in the linked list. It should contain an integer ‘data‘ field and a pointer ‘next‘ to the next node.
- A structure ‘Stack‘ to represent the stack itself. It should contain a pointer ‘top‘ to the top node of the stack.

c)

Write a function called ‘createStack‘ that allocates memory for a Stack structure, initializes the top pointer to NULL, and returns a pointer to the created Stack. Return NULL if memory allocation fails.

```c
// complete this
Stack* createStack() {
    // ...
}
```

d)

Write a function called ‘push‘ that takes a pointer to the ‘Stack‘ and an integer ‘data‘ as input. It should create a new ‘StackNode‘, store the data in it, and add it to the top of the stack. Handle potential memory allocation failure for the new node. If allocation fails, the stack should remain unchanged, and the function should indicate failure by returning -1. Return 1 on success.

```
// complete this
int push(...) {
 // ...
}
```

e)

Write a function called ‘pop‘ that takes a pointer to the Stack as input. It should remove the top node from the stack, free the memory allocated for that node, and return the data stored in it. If the stack is empty, it should indicate an error by printing a relevant statement and returning -1.

```
// complete this
int pop(...) {
 // ...
}
```

f)

Write a C function ‘freeStack‘ that takes a pointer to the ‘Stack‘ as input. It should free all the nodes remaining in the stack and finally free the ‘Stack‘ structure itself. After this function call, the stack pointer passed to it should not be used anymore.

```
// complete this
void freeStack(...) {
    // ...
}
```